

# Esame Programmazione di Sistema / API Programming

Data: 2022-09-08

## QUESTIONS

### Domanda 1

Si definiscano le principali aree di memoria associate ad un eseguibile e si mostri, attraverso opportuni esempi di codice, in quale situazione ciascuna di esse viene utilizzata.

### Domanda 2

Sia dato un primo valore di tipo `std::cell::Cell<T>` ed un secondo valore di tipo `std::cell::RefCell<T>` (dove `T` fa riferimento alla medesima entità). Si indichino le differenze tra i due e le modalità di occupazione della memoria (quantità, zone di memoria, ecc.).

### Domanda 3

In un programma che utilizza una sincronizzazione basata su Condition Variable, è possibile che alcune notifiche vengano perse? Se sì, perché? In entrambi i casi si produca un esempio di codice che giustifichi la risposta.

### Domanda 4

In un sistema concorrente, ciascun thread può pubblicare eventi per rendere noto ad altri thread quanto sta facendo.

Per evitare un accoppiamento stretto tra mittenti e destinatari degli eventi, si utilizza un Dispatcher: questo è un oggetto thread-safe che offre il metodo `dispatch(msg: Msg)` mediante il quale un messaggio di tipo generico `Msg` (soggetto al vincolo di essere clonabile) viene reso disponibile a chiunque si sia sottoscritto. Un thread interessato a ricevere messaggi può invocare il metodo `subscribe()` del Dispatcher: otterrà come risultato un oggetto di tipo `Subscription` mediante il quale potrà leggere i messaggi che da ora in poi saranno pubblicati attraverso il Dispatcher. Per ogni sottoscrizione attiva, il Dispatcher mantiene internamente l'equivalente di una coda ordinata (FIFO) di messaggi non ancora letti. A fronte dell'invocazione del metodo `dispatch(msg:Msg)`, il messaggio viene clonato ed inserito in ciascuna delle code esistenti.

L'oggetto `Subscription` offre il metodo bloccante `read() -> Option<Msg>`: se nella coda corrispondente è presente almeno un messaggio, questo viene rimosso e restituito; se nella coda non è presente nessun messaggio e il Dispatcher esiste ancora, l'invocazione si blocca fino a che non viene inserito un nuovo messaggio; se invece il Dispatcher è stato distrutto, viene restituito il valore corrispondente all'opzione vuota.

Gli oggetti `Dispatcher` e `Subscription` sono in qualche modo collegati, ma devono poter avere cicli di vita indipendenti: la distruzione del Dispatcher non deve impedire la consumazione dei messaggi già recapitati ad una Subscription, ma non ancora letti; parimenti, la distruzione di una

Subscription non deve impedire al Dispatcher di consegnare ulteriori messaggi alle eventuali altre Subscription presenti.

Si implementino le strutture dati Dispatcher e Subscription, a scelta, nel linguaggio Rust o C++11.

# SOLUTIONS

## Domanda 1

Le principali aree di memoria associate ad un eseguibile sono:

- **Text/Code Segment:** Contiene le istruzioni macchina del programma compilato. È tipicamente di sola lettura per prevenire modifiche accidentali al codice durante l'esecuzione.
- **Data Segment:** Contiene le variabili globali e statiche che sono esplicitamente inizializzate nel codice.
- **BSS Segment:** Contiene variabili globali e statiche non inizializzate (vengono inizializzate a zero dal sistema operativo).
- **Stack:** Area di memoria utilizzata per l'allocazione statica (a tempo di compilazione si conosce la dimensione). Cresce e decresce automaticamente con le chiamate a funzione (LIFO). Memorizza le variabili locali, i parametri delle funzioni e gli indirizzi di ritorno. In Rust, le variabili di default sono allocate sullo stack.
- **Heap:** Area di memoria dinamica utilizzata per allocare dati la cui dimensione è nota solo a runtime o che devono sopravvivere al termine della funzione in cui sono creati. L'allocazione non è contigua e il programmatore o costrutti appositi (smart pointers come `Box`, `Rc`, `Arc` in Rust) ne gestiscono il ciclo di vita.

### Esempio in Rust:

```
1 // DATA SEGMENT: Variabile statica
2 static SALUTO: &str = "Ciao Mondo";
3
4 fn main() {
5     // STACK: le variabili locali e a dimensione fissa sono allocate sullo stack.
6     let x: i32 = 42;
7     let array: [i32; 3] = [1, 2, 3];
8
9     // HEAP: L'allocazione di tipi dinamici o esplicitamente in memoria dinamica
10    // 'vettore' (la struttura dati Vec) è allocato sullo stack (come puntatore,
11    // lunghezza e capacità), ma i suoi elementi effettivi sono allocati sull'
    heap.
12    let mut vettore: Vec<i32> = Vec::new();
13    vettore.push(1);
14    vettore.push(2);
15
16    // BOX: alloca esplicitamente il valore sull'heap
17    let boxed_val = Box::new(100);
18 }
```

## Domanda 2

Entrambi i tipi `Cell<T>` e `RefCell<T>` in Rust sono impiegati per la *interior mutability*, consentendo di mutare i dati anche quando ci sono riferimenti immutabili al contenitore esterno.

### Differenze e Regole d'accesso:

- **Cell<T>:** Gestisce l'accesso fornendo metodi che spostano (`move`) o copiano (`copy`) il valore (es. `get()` o `set()`). Non consente mai di ottenere riferimenti (`&T` o `&mut T`) ai dati interni. È tipicamente usato per tipi di base o strutture che implementano il trait `Copy`.

- `RefCell<T>`: Consente di ottenere riferimenti (`borrow()` o `borrow_mut()`) ai dati interni, spostando i controlli del *borrow checker* da tempo di compilazione a *runtime*. Se le regole vengono violate (ad es. due riferimenti mutabili contemporanei), il programma andrà in `panic`.

### Occupazione in Memoria:

- `Cell<T>`: Occupa esattamente lo stesso spazio di memoria del tipo `T` che incapsula (non c'è alcun overhead logico aggiuntivo). È garantito che ha la stessa rappresentazione in memoria di `T`.
- `RefCell<T>`: Occupa lo spazio di `T` più un contatore aggiuntivo (`Cell<isize>`) per tenere traccia a runtime del numero di *borrow* attivi e mutabili. Di conseguenza, la dimensione in memoria di `RefCell<T>` è `size_of::T() + size_of::isize()` (più eventuale padding per l'allineamento dei dati).

Entrambi, se istanziati localmente e non all'interno di un costrutto apposito come `Box` o `Rc`, risiedono interamente nello **stack**.

## Domanda 3

Sì, è assolutamente possibile che le notifiche di una Condition Variable (CV) vadano perse (il cosiddetto problema della *lost wakeup*). Questo accade tipicamente quando il thread notificatore emette il segnale (tramite `notify_one` o `notify_all`) *prima* che il thread ricevente si sia effettivamente messo in attesa sulla CV, e l'evento in questione non è memorizzato in una variabile di stato visibile. Inoltre, le CV possono essere soggette a *spurious wakeups* (risvegli senza che la CV sia stata effettivamente notificata).

Per risolvere questi problemi, è **necessario** associare un predicato (una variabile di stato) protetto dallo stesso Mutex usato per la CV e verificare lo stato prima e dopo il risveglio. In Rust questo si ottiene col costrutto `wait_while`.

### Esempio di anti-pattern (Lost Wakeup potenziale):

```
1 // Thread 1
2 let mut guard = mutex.lock().unwrap();
3 // Se Thread 2 esegue la notifica PRIMA che Thread 1 esegua la riga
4 // seguente, la notifica viene ignorata e Thread 1 aspetterà' per sempre.
5 guard = cv.wait(guard).unwrap();
6
7 // Thread 2
8 cv.notify_one();
```

### Soluzione corretta (Uso dello stato e `wait_while`):

```
1 use std::sync::{Arc, Mutex, Condvar};
2 use std::thread;
3
4 fn main() {
5     let pair = Arc::new((Mutex::new(false), Condvar::new()));
6     let pair_clone = Arc::clone(&pair);
7
8     // Thread 1 (Waiting)
9     let t1 = thread::spawn(move || {
10         let (lock, cvar) = &*pair_clone;
11         let ready = lock.lock().unwrap();
12         // wait_while si blocca e rilascia il lock solo se il controllo fallisce,
```

```

13         // ritentando in caso di risvegli. Se il flag e' gia' true, non si blocca.
14         let _ready = cvar.wait_while(ready, |ready_flag| !*ready_flag).unwrap();
15         println!("Condizione verificata, proseguendo!");
16     });
17
18     // Thread 2 (Notifying)
19     let t2 = thread::spawn(move || {
20         let (lock, cvar) = &*pair;
21         let mut ready = lock.lock().unwrap();
22         // Aggiorna lo stato (predicato) e notifica
23         *ready = true;
24         cvar.notify_one();
25     });
26
27     t1.join().unwrap();
28     t2.join().unwrap();
29 }

```

## Domanda 4

### Implementazione in Rust

I canali `mpsc` della libreria standard `std::sync::mpsc` si adattano perfettamente alle specifiche richieste. I canali sono per natura asimmetrici (*multiple producers, single consumer*) ma possiedono proprietà di ciclo di vita ideali: se tutti i **Sender** vengono distrutti, il **Receiver** terminerà le chiamate pendenti di `recv()` restituendo errore, traducibile fluidamente in un `None`. I messaggi inviati restano nel buffer del **Receiver** anche se i **Sender** vengono distrutti.

Per gestire la lista dei **Sender** all'interno del metodo `subscribe(&self)`, abbiamo bisogno di *interior mutability* fornita da `Mutex`. Inoltre, per non trattenere in memoria invii superflui verso `Subscription` terminate, utilizziamo il metodo `retain()` che filtra e rimuove dal vettore tutti i **Sender** che non riescono più ad inviare messaggi.

```

1 use std::sync::mpsc::{channel, Receiver, Sender};
2 use std::sync::Mutex;
3
4 /// Struttura Dispatcher
5 pub struct Dispatcher<Msg> {
6     // Vettore protetto da Mutex per supportare interior mutability
7     // e accessi sicuri da piu thread.
8     senders: Mutex<Vec<Sender<Msg>>>,
9 }
10
11 /// Struttura Subscription
12 pub struct Subscription<Msg> {
13     receiver: Receiver<Msg>,
14 }
15
16 impl<Msg: Clone> Dispatcher<Msg> {
17     /// Inizializza un nuovo Dispatcher
18     pub fn new() -> Self {
19         Dispatcher {
20             senders: Mutex::new(Vec::new()),
21         }
22     }
23
24     /// Sottoscrive un nuovo client e restituisce l'oggetto Subscription.
25     pub fn subscribe(&self) -> Subscription<Msg> {

```

```

26     // Creo una coppia di canali asincroni mpsc.
27     let (tx, rx) = channel();
28
29     // Registro il mittente nel dispatcher.
30     self.senders.lock().unwrap().push(tx);
31
32     Subscription { receiver: rx }
33 }
34
35 /// Esegue il dispatch del messaggio a tutte le subscription attive.
36 pub fn dispatch(&self, msg: Msg) {
37     let mut senders = self.senders.lock().unwrap();
38
39     // Usiamo 'retain' per mantenere solo i Sender validi.
40     // tx.send(...) fallisce se il Receiver (nella Subscription)
41     // e' stato distrutto. Se l'invio ha successo, is_ok() restituisce true
42     // e retain mantiene il Sender nel Vettore. Altrimenti viene rimosso.
43     senders.retain(|tx| tx.send(msg.clone()).is_ok());
44 }
45 }
46
47 impl<Msg> Subscription<Msg> {
48     /// Legge il prossimo messaggio. Si blocca se non ci sono messaggi.
49     /// Ritorna None se il Dispatcher e' stato distrutto.
50     pub fn read(&self) -> Option<Msg> {
51         // recv() attende in blocco finche' c'e' un messaggio o tutti
52         // i Sender ad esso collegati sono stati distrutti.
53         self.receiver.recv().ok()
54     }
55 }

```